

Шифрование различных исходных текстов одним ключом

Лабораторная работа 8

Смольняков Данил Евгеньевич

Основы информационной безопасности

Цель работы

Цель

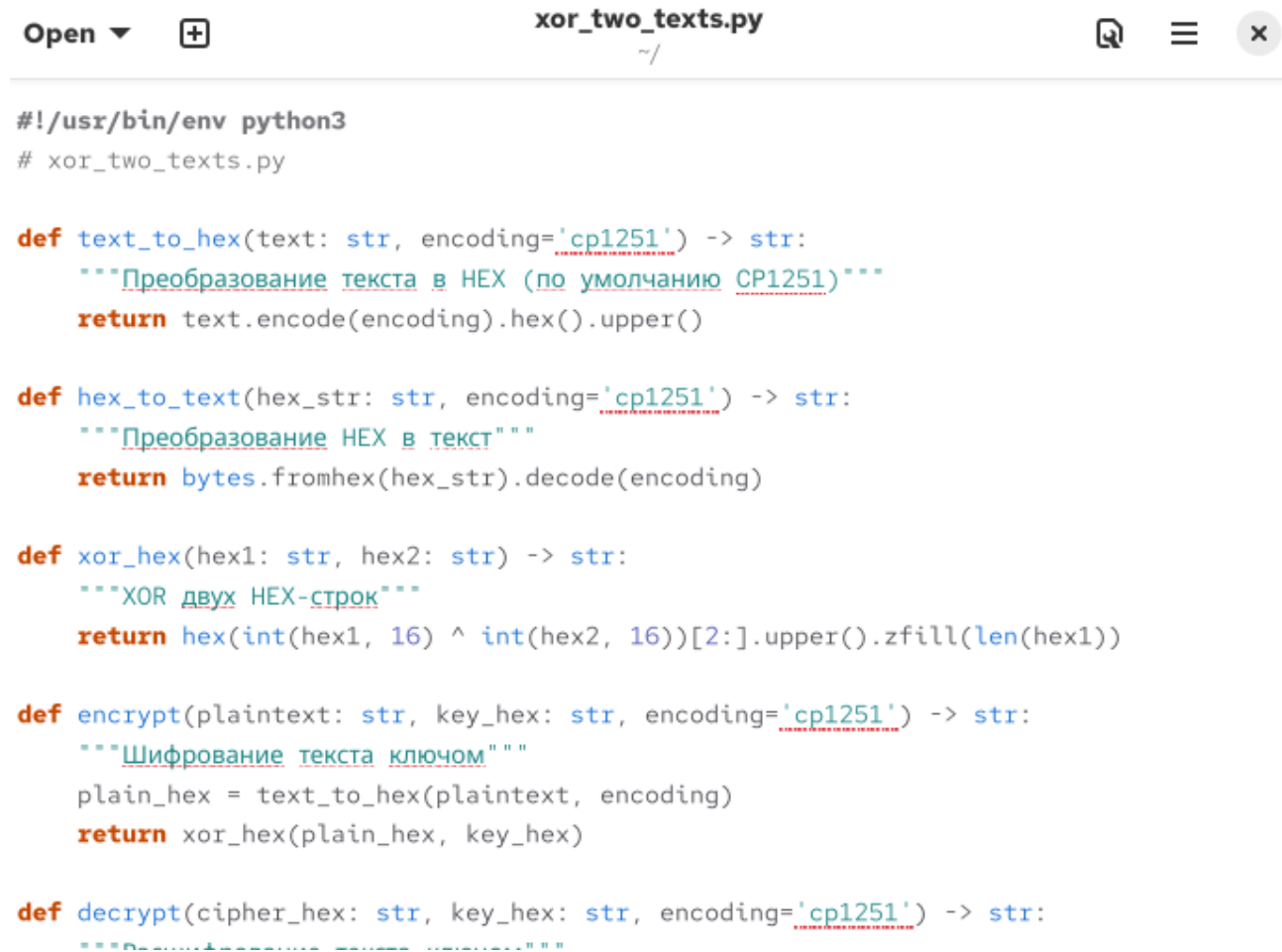
- Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом
 - Научиться восстанавливать один из текстов, зная другой текст и оба шифротекста (без знания ключа)
-

Скриншоты и описание действий

1. Код программы (xor_two_texts.py)

- **Действие:** Разработка приложения на Python для шифрования двух текстов одним ключом

- Скриншот:



```
#!/usr/bin/env python3
# xor_two_texts.py

def text_to_hex(text: str, encoding='cp1251') -> str:
    """Преобразование текста в HEX (по умолчанию CP1251)"""
    return text.encode(encoding).hex().upper()

def hex_to_text(hex_str: str, encoding='cp1251') -> str:
    """Преобразование HEX в текст"""
    return bytes.fromhex(hex_str).decode(encoding)

def xor_hex(hex1: str, hex2: str) -> str:
    """XOR двух HEX-строк"""
    return hex(int(hex1, 16) ^ int(hex2, 16))[2:].upper().zfill(len(hex1))

def encrypt(plaintext: str, key_hex: str, encoding='cp1251') -> str:
    """Шифрование текста ключом"""
    plain_hex = text_to_hex(plaintext, encoding)
    return xor_hex(plain_hex, key_hex)

def decrypt(cipher_hex: str, key_hex: str, encoding='cp1251') -> str:
    """Дешифрование текста ключом"""
```

Код программы

2. Запуск программы с исходными данными

- Действие: Шифрование двух текстов P1 и P2 одним ключом K

- **Скриншот:**

```

desmoljnyakov@vbox:~$ python3 xor_two_texts.py
=====
Лабораторная работа №8
Шифрование двух текстов одним ключом
=====

Исходные данные:
P1 = НаВашисходящийот1204
P2 = ВСеверныйфилиалБанка
K = 050C177F0E4E37D29410092E2257FFC80BB27054

Шифротексты:
C1 = C8ECD59FF6A6C6277AF4F6D7CABE113A3A804060
C2 = C7DDF29DEBBEDA297DE4E1C5CAB71409EB5F9AB4

Расшифровано:
P1 = НаВашисходящийот1204
P2 = ВСеверныйфилиалБанка

Восстановление P2 без ключа:
C1 ⊕ C2 ⊕ P1 = ВСеверныйфилиалБанка

Проверка свойства: C1 ⊕ C2 = P1 ⊕ P2
C1 ⊕ C2 = 0F3127021D181C0E0710171200090533D1DFDAD4
P1 ⊕ P2 = 0F3127021D181C0E0710171200090533D1DFDAD4
Равенство выполнено: True
desmoljnyakov@vbox:~$ █

```

Запуск программы с исходными данными

3. Восстановление второго текста без ключа

- **Действие:** Используя формулу $C1 \oplus C2 \oplus P1 = P2$, восстановлен текст P2 без знания ключа

- **Скриншот:**

```
print("4. ВОССТАНОВЛЕНИЕ P2 БЕЗ ЗНАНИЯ КЛЮЧА (СКРИНШОТ 3)")
print("=" * 70)
print("    Формула: C1 ⊕ C2 ⊕ P1 = P2")

recovered_P2 = recover_second_text(C1, C2, P1)
print(f"\n    Известно: P1 = {P1}")
print(f"    Известно: C1 = {C1}")
print(f"    Известно: C2 = {C2}")
print(f"\n    Вычисляем: C1 ⊕ C2 = {xor_hex(C1, C2)}")
print(f"    Вычисляем: (C1⊕C2) ⊕ P1 = {xor_hex(xor_hex(C1, C2),
text_to_hex(P1))}")
print(f"\n    Результат: P2 = {recovered_P2}")
```

Восстановление второго текста без ключа

4. Проверка свойства $C1 \oplus C2 = P1 \oplus P2$

- **Действие:** Доказано, что операция XOR сокращает ключ: $C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K) = P1 \oplus P2$
- **Скриншот:**

```
# ===== ПРОВЕРКА СВОЙСТВА C1⊕C2 = P1⊕P2 (СКРИНШОТ 4) =====
print("\n" + "=" * 70)
print("5. ПРОВЕРКА СВОЙСТВА: C1 ⊕ C2 = P1 ⊕ P2 (СКРИНШОТ 4)")
print("=" * 70)
print("    Математически: (P1⊕K) ⊕ (P2⊕K) = P1⊕P2 (ключ сокращается)\n")

c1_xor_c2 = xor_hex(C1, C2)
p1_xor_p2 = xor_hex(text_to_hex(P1), text_to_hex(P2))

print(f"    C1 ⊕ C2 = {c1_xor_c2}")
print(f"    P1 ⊕ P2 = {p1_xor_p2}")
print(f"\n    Равенство выполнено: {c1_xor_c2 == p1_xor_p2} ✓")
```

Проверка свойства $C1 \oplus C2 = P1 \oplus P2$

5. Итеративное восстановление текста

- **Действие:** Демонстрация пошагового восстановления, когда известна только часть текста-шаблона

=====

4. ВОССТАНОВЛЕНИЕ P2 БЕЗ ЗНАНИЯ КЛЮЧА (СКРИНШОТ 3)

=====

Формула: $C1 \oplus C2 \oplus P1 = P2$

Известно: $P1 = \text{НаВашисходящийот1204}$

Известно: $C1 = \text{C8ECD59FF6A6C6277AF4F6D7CABE113A3A804060}$

Известно: $C2 = \text{C7DDF29DEBBEDA297DE4E1C5CAB71409EB5F9AB4}$

Вычисляем: $C1 \oplus C2 = \text{0F3127021D181C0E0710171200090533D1DFDAD4}$

Вычисляем: $(C1 \oplus C2) \oplus P1 = \text{C2D1E5E2E5F0EDFBE9F4E8EBE8E0EBC1E0EDEAE0}$

Результат: $P2 = \text{ВСеверныйфилиалБанка}$

=====

5. ПРОВЕРКА СВОЙСТВА: $C1 \oplus C2 = P1 \oplus P2$ (СКРИНШОТ 4)

=====

Математически: $(P1 \oplus K) \oplus (P2 \oplus K) = P1 \oplus P2$ (ключ сокращается)

$C1 \oplus C2 = \text{0F3127021D181C0E0710171200090533D1DFDAD4}$

$P1 \oplus P2 = \text{0F3127021D181C0E0710171200090533D1DFDAD4}$

Равенство выполнено: True ✓

=====

6. ИТЕРАТИВНОЕ ВОССТАНОВЛЕНИЕ (СКРИНШОТ 5)

=====

Если известна только ЧАСТЬ текста P1, можно восстановить соответствующую часть текста P2:

Известно 'На' → восстанавливаем 'ВС'

Известно 'НаВаши' → восстанавливаем 'ВСевер'

Известно 'НаВашисход' → восстанавливаем 'ВСеверныйф'

Известно 'НаВашисходящий' → восстанавливаем 'ВСеверныйфилиа'

Известно 'НаВашисходящийот12' → восстанавливаем 'ВСеверныйфилиалБан'

Полное восстановление P2: ВСеверныйфилиалБанка

Оригинальный P2: ВСеверныйфилиалБанка

Совпадение: True ✓

- **Скриншот:**
Итеративное восстановление текста

Контрольные вопросы

1. Как, зная один из текстов (P1 или P2), определить другой, не зная при этом ключа?

Используя свойство XOR: $C1 \oplus C2 = P1 \oplus P2$. Зная P1, C1 и C2, можно найти $P2 = C1 \oplus C2 \oplus P1$

2. Что будет при повторном использовании ключа при шифровании текста?

При повторном использовании ключа теряется абсолютная стойкость шифра. Если один и тот же ключ используется для шифрования двух разных сообщений, злоумышленник может получить XOR этих сообщений: $P1 \oplus P2 = C1 \oplus C2$. Зная или угадав одно из сообщений, он восстановит второе.

3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов?

Два открытых текста P1 и P2 одинаковой длины шифруются одним и тем же ключом K:

$$C1 = P1 \oplus K$$

$$C2 = P2 \oplus K$$

Шифротексты C1 и C2 передаются получателю.

4. Перечислите недостатки шифрования одним ключом двух открытых текстов

Компрометация текстов: Если один текст становится известен, второй легко восстанавливается

Потеря абсолютной стойкости: Шифр перестаёт быть абсолютно стойким

Возможность криптоанализа: Злоумышленник получает $P1 \oplus P2$, что даёт информацию для статистического анализа

Требование одинаковой длины: Тексты должны быть одной длины (или ключ должен быть не короче каждого текста)

5. Перечислите преимущества шифрования одним ключом двух открытых текстов

Экономия ключевого материала: Не нужно генерировать отдельный ключ для каждого сообщения

Простота реализации: Та же операция XOR для обоих сообщений

Скорость: Нет дополнительных вычислительных затрат

Симметричность: Шифрование и расшифрование выполняются одинаково

Заключение

- В ходе выполнения лабораторной работы было разработано приложение на Python, реализующее шифрование двух текстов одним ключом в режиме однократного гаммирования.
- Экспериментально подтверждено математическое свойство: $C1 \oplus C2 = P1 \oplus P2$.
- Продемонстрирована возможность восстановления второго текста без знания ключа, используя формулу $P2 = C1 \oplus C2 \oplus P1$.

- Показано, что повторное использование ключа для разных сообщений приводит к потере абсолютной стойкости шифра.
- Получены практические навыки работы с HEX-представлением текстов и операцией XOR в криптографии.